Decision Orchestration: Learning Annotation Procedures Five Sentences at a Time

Anonymous ACL submission

Abstract

A new annotation scheme, language or domain usually starts with a guideline and little annotated data. We propose *decision orchestration*: the guideline is written as an executable procedure in which rules enumerate candidates and assemble the output, and a decision model answers only closed questions, returning a probability for each option the guideline allows. The output is well-formed by construction, every decision is cheap and has a confidence, and uncertain decisions can be routed to a stronger model. We learn the procedure, not model weights: an LLM reflector reads five annotated sentences at a time, with the gold and predicted annotations and every question asked, and proposes edits to the procedure's rules, options and thresholds; each edit is kept only if it measurably improves the score. On verbal multiword expression identification (PARSEME), a procedure learned from 60 annotated sentences reaches 46.3 F in English, 13 points above the best system of the 2018 shared task, which was trained on 3,471 sentences, and 57.8 F in Chinese, 18 points above an LLM annotating directly with ten gold examples; reflection costs under one dollar per language. On Uniform Meaning Representation parsing, where the output is mostly graph structure, closed decisions alone do not beat direct generation, although reflection adapts a Chinese procedure to English (+4.1 F) and routing adds 2.5 F. Decision orchestration pays off where a guideline is a sequence of tests over enumerable candidates.

1 Introduction

A new annotation scheme, or an old one in a new language or domain, usually starts with a written guideline and very little annotated data. Training a supervised model needs thousands of examples; prompting an LLM to write the annotation needs none, but hands the model the whole job: deciding what to annotate, applying the scheme's conventions, and producing well-formed output, all at once and at the price of a generative model for every sentence.

Most of a guideline is not judgment. It tells the annotator where to look, which units to consider, which tests to apply in which order, and how to write the result; only some steps require understanding the sentence. We propose **decision orchestration** (Figure 1): write the guideline as an executable *procedure* in which rules enumerate candidates and assemble the output, and a *decision model* answers only closed questions, each a choice among options taken from the guideline, with a probability distribution. The output is well-formed by construction. Every decision is cheap, inspectable and comes with a confidence. Uncertain decisions can be routed to a stronger model.

A hand-written procedure encodes the guideline as its author read it. Gold annotations encode the conventions annotators actually followed, and the two differ. We therefore *learn the procedure*, not model weights. An LLM reflector reads five training sentences at a time, with the gold and predicted annotations and every question asked with its answer, and proposes small edits to the procedure's code, options and thresholds. Each edit is kept only if it improves the score on the batch, on earlier batches and on a development set. The procedure's history becomes a sequence of small tested diffs that a person can read, and the whole learning process costs a few dollars.

We test the method on two schemes that differ in how much of the output is structure: verbal multiword expressions (PARSEME), where an annotation is a set of categorized token groups, and Uniform Meaning Representation (UMR), where it is a graph. Each is tested in Chinese and English. Our findings:

- On PARSEME, a procedure learned from **60 annotated sentences** reaches 46.3 F on English, **13 points above the best system of the 2018**

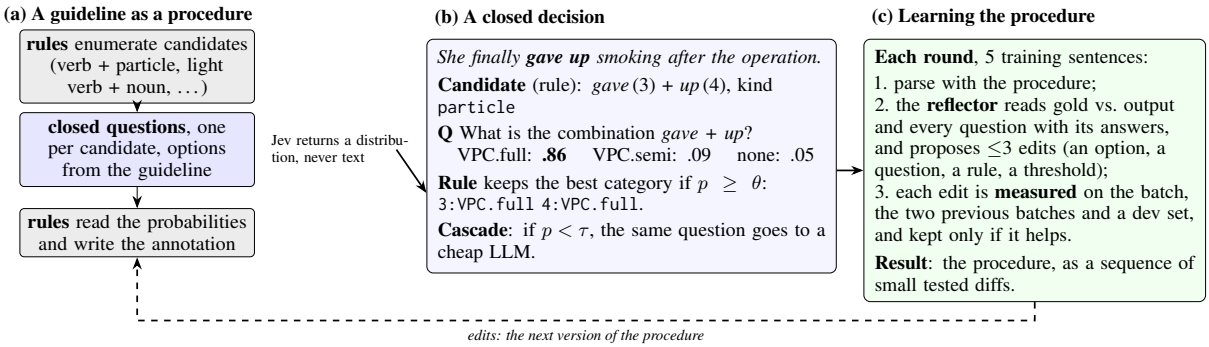


Figure 1: Decision orchestration. (a) A guideline is written as a procedure in which rules enumerate candidates and assemble the output, and a decision model answers closed questions with probabilities. (b) One closed decision in the verbal-multiword-expression procedure. (c) The procedure is learned from a few annotated sentences at a time: an LLM proposes edits, and each edit is kept only if it measurably helps.

shared task, which trained on all 3,471 training sentences, and 57.8 F on Chinese, **18 points above** Claude Haiku annotating directly with ten gold examples. Twelve rounds of reflection add 5.5 F on Chinese (7.8 on unseen expressions) for $0.53 of reflector calls, and the decisions for the whole Chinese test set cost $0.06.

- On UMR, where the output is mostly structure, closed decisions alone do not beat direct generation. Routing uncertain decisions adds 2.5 F in Chinese, and reflection adapts a procedure written for Chinese to English (+4.1 F); but direct generation with ten examples matches the best procedure in Chinese and exceeds it by 16 F in English.

- The procedure's options must follow the annotation's conventions, not the full inventory of the guideline: a UMR procedure that offered the decision model the 27-value aspect lattice of the guidelines, where the gold uses seven coarse values, gained 5.2 F when its aspect options were restricted to those seven and one polarity rule was restored.

Decision orchestration thus pays off where a guideline is a sequence of tests over enumerable candidates, and it tells us, through the measured edits and the routed decisions, where a scheme is not.

2 Related Work

LLMs as annotators. LLMs can label data directly (Wang et al., 2021; Gilardi et al., 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024), and human–LLM collaboration routes uncertain items to people (Li et al., 2023; Wang et al., 2024). GoLLIE (Sainz et al., 2024) fine-tunes a model to follow guidelines written as code, Code4Struct (Wang et al., 2023) writes event structures as code, and guideline learning (Pang et al., 2023) induces guideline text from errors. These keep the model in charge of writing the annotation. We keep the guideline's procedure in code and ask the model only closed questions inside it, so the output is well-formed by construction and every decision has a probability.

LM programs and structured output. Prompt chains and decomposed prompting compose model calls (Wu et al., 2022; Khot et al., 2023); programs can call models or declare their constraints (Gao et al., 2023; Gupta and Kembhavi, 2023; Beurer-Kellner et al., 2023; Khattab et al., 2024), and grammar-constrained decoding controls output form (Willard and Louf, 2023; Geng et al., 2023), possibly at a cost to reasoning (Tam et al., 2024). Agents let the model choose the next step (Yao et al., 2023). In our procedures the control flow is fixed code, and the model's only output is a distribution over listed options.

Optimizing prompts and programs. Prompts and programs can be optimized from feedback (Zhou et al., 2023; Pryzant et al., 2023; Yang et al., 2024; Opsahl-Ong et al., 2024; Yuksekgonul et al., 2025), by reflecting on execution traces (Agrawal et al., 2026) or by growing a playbook of lessons (Zhang et al., 2026); workflows and scaffolds can be searched as well (Zhang et al., 2025; Hu et al., 2025; Zelikman et al., 2024). Self-feedback without an external signal is unreliable (Huang et al., 2024). Our reflector reads gold-aligned traces of a few sentences and edits code, options and thresholds; every edit is kept only if it measurably helps.

The result resembles transformation-based learning (Brill, 1995) and data programming (Ratner et al., 2017) in being a readable artifact, but its rules are written by an LLM and checked by measurement.

Calibration and cascades. Calibration and selective prediction study when to trust a model's confidence (Guo et al., 2017; Jiang et al., 2021; Zhao et al., 2021; Kamath et al., 2020; Kadavath et al., 2022); cascades escalate inputs to costlier models (Chen et al., 2024). We escalate individual closed questions inside a structured annotation, by the decision model's own probability.

The two tasks. PARSEME systems for verbal multiword expressions are mostly supervised sequence taggers or graph-based parsers trained on the full training data, often on top of pretrained encoders (Ramisch et al., 2018, 2020); the 2.0 edition also reports instruction-tuned LLMs (Scholivet et al., 2026). Meaning representation parsers likewise learn from large corpora (Bevilacqua et al., 2021; Zhou et al., 2021; Bai et al., 2022), which UMR does not yet have (Van Gysel et al., 2021; Bonn et al., 2023, 2024, 2025); Chun and Xue (2024) convert other representations, and Markle et al. (2025) propose a parsing setup for UMR. We use both tasks as tests of a method for learning annotation procedures from little data, not to compete with systems trained on full corpora.

3 Decision Orchestration

Setting. An annotation scheme maps an input x (a sentence, or a document) to a structured annotation y: a set of typed token groups, a graph, a set of relations. In the cold start we have the scheme's prose guideline, the external resources it refers to (e.g., a frame lexicon), an automatic scorer, and a few annotated sentences. We train no model weights. We write, and then learn, a *procedure* Π that maps x to y.

3.1 Procedures: rules around closed decisions

A guideline is mostly a procedure: it says where to look, which units to consider, which tests to apply in which order, and how to write the result. Only some steps need judgment. Π makes this split explicit (Figure 1). It consists of three parts.

- **Rules** enumerate candidates from the input (words, spans, verb–object pairs, the senses of a verb in a lexicon, the mentions before a pronoun) and decide what is decidable without judgment (number formats, function words, the assembly of the output).
- **Closed questions** cover the rest. A question names its subject in the input and lists its options, each with a one-line description taken from the guideline: a choice ("which category is *give up* here: VPC.full, VPC.semi, none"), a yes/no question, or a graded score. Questions are asked in rounds; a round may depend on the answers of the previous one.
- **Parameters** θ, such as the probability an option needs before a rule acts on it.

A decision model answers every question of a round at once with a probability distribution over its options and no text. Rules read the distributions against θ and assemble y. The output is well-formed by construction, since only rules write it, and every decision carries a probability that can be inspected, thresholded or routed. In our implementation a procedure is a folder of Python files with their question texts and option lists, which makes it readable and editable by a person and by a language model alike.

Decision model. We use Jev (TypeSafe, 2026), a model served behind a typed closed-question interface (choice, yes/no, score) that returns only probabilities. The interface fixes what the model can do: it never writes structure, so a procedure is the only place where the annotation's form is defined. Decisions are cheap. At the list price we used, $0.02 per million tokens, a decision costs a few hundred-thousandths of a dollar. Answers are cached by request, so re-running a procedure only pays for the questions that changed.

3.2 Learning the procedure in small batches

A hand-written procedure encodes the guideline as its author read it; the gold data encode the conventions annotators actually followed. We close the gap by editing the procedure, never model weights, a few sentences at a time (Algorithm 1). Each round r takes the next batch B_r of five training sentences and parses it with the current procedure. A *reflector*, a general LLM (Claude Sonnet 5.5), reads a report of the batch: the gold and predicted annotations, the scorer's findings, and, for every node, the questions asked and the distributions Jev returned. It sees the procedure's editable files and the titles of edits tried in earlier rounds, diagnoses the most costly differences and proposes at most

Algorithm 1 Measured reflection in small batches.

Require: seed procedure Π_0, training pool P, development set D, batch size b=5

for $r = 1, \ldots, R$ **do**
  $B \leftarrow$ next b sentences of P; $G \leftarrow$ batches of rounds $r-2, r-1$
  $s \leftarrow \textsc{Score}(\Pi, \{B, G, D\})$
  $E \leftarrow \textsc{Reflect}(\Pi, \textsc{Report}(\Pi, B))$ ▷ ≤ 3 edits
  for $e \in E \cup \textsc{Revise}(\text{rejected})$ **do**
    $s' \leftarrow \textsc{Score}(e(\Pi), \{B, G, D\})$
    if $\bar{s}' - \bar{s} \geq 0.3 \wedge s'_D - s_D \geq -0.5 \wedge s'_G - s_G \geq -1$ **then**
      $\Pi \leftarrow e(\Pi)$; $s \leftarrow s'$
    end if
  end for
end for

three edits, each an exact find-and-replace in one file: a new option with the rule branch that uses it, a new question where a rule guessed, a changed threshold, a convention written into a rule, or a clarified option description.

Every edit is *measured* before it is kept. It is applied on top of the edits already accepted in the round, and the procedure is re-run on three sets: the batch, a regression set (the batches of the two previous rounds) and a fixed development set. The edit stays if the mean F over the three rises by at least 0.3, development F does not fall by more than 0.5, regression F by no more than 1.0, and no new runtime errors appear. Rejected edits go back to the reflector once, with their measured effect on each set, to be narrowed or rewritten. Because Jev's answers are cached, a trial pays only for the questions that the edit changes; a whole round costs cents in decisions and a few cents in reflector calls.

Small batches keep the reflector's context short and its diagnosis specific, and they make the procedure's history a sequence of small, attributable, tested changes. The regression and development sets keep the procedure from overfitting to the batch; we find that a development set of ten sentences is too small to do so reliably (§7).

3.3 Routing uncertain decisions

Each decision comes with a probability, so the procedure knows which of its decisions are uncertain. In the *cascade*, a question whose top option has probability below τ is asked again of a cheap generative model (Claude Haiku 5.5), given the same input and the same question and options; its answer replaces Jev's. The procedure itself is unchanged, and only uncertain questions pay the higher price.

4 Two Annotation Tasks as Procedures

We test the method on two annotation schemes that differ in what the output is and how much of it is structure. In both, the seed procedure was written by hand from the published guideline, and the same driver, reflector and decision model learn it.

Verbal multiword expressions (PARSEME). The PARSEME shared tasks annotate verbal multiword expressions (VMWEs): sets of tokens, possibly discontinuous, with a category such as a verbal idiom (VID), a light-verb construction (LVC.full, LVC.cause), a verb–particle construction (VPC.full, VPC.semi), an inherently adpositional verb (IAV), or, for Chinese, a multi-verb construction (MVC) (Ramisch et al., 2018, 2020). In Chinese a VMWE is often a single token: a verb with its directional or resultative complement written as one word (推出, 放入) or a four-character idiom. We use Chinese from edition 1.2 and English from edition 1.1, the last edition with English. The input gives each token's lemma, part of speech and dependency relation; these are part of the data, and the procedure reads them like any other input.

The seed procedure has two files. `candidates.py` proposes candidates by rule: in Chinese, a verb token ending in a complement (split into stem and complement), a verb and a following complement token, a light verb and its object, and a four-character token; in English, a verb and its particle, a light verb and its object noun, a verb and the preposition after it, and *make/let/get/break* with a bare complement. `decide.py` asks one choice question per candidate, whose options are the categories the candidate's kind allows plus *none*, each described in one line from the guideline; a candidate is kept when its best option is a category with probability ≥ 0.5, and overlaps are resolved by probability. The seed's candidates cover about three quarters of the gold VMWEs of the training data. All questions of a sentence go to Jev in one request.

Uniform Meaning Representation (UMR). UMR (Bonn et al., 2023, 2025) represents each sentence as a rooted graph of concepts (PropBank senses for events, words or entity types for objects, abstract predicates such as `identity-91`) linked by roles, with attributes such as aspect and polarity, plus document-level temporal, modal and coreference relations. The output is mostly structure. The procedure splits a sentence top-down. A routing

skill decides whether a span is coordinated clauses, a clause, a noun phrase, a named entity or a leaf; a clause skill asks for the predicate, its sense, aspect, modal strength and negation, and the role of every word in the clause; noun-phrase and entity skills ask for the head, the shape and the modifiers' relations. Unexpanded parts of a fragment become placeholders that go back to routing, rules cover words that no child took, and a reentrancy pass merges coreferent nodes. A sentence takes 11–49 requests to Jev, depending on the procedure's version. We start from the closed procedure of the earlier version of this work and from its recent revision, whose questions and options were moved into data files and whose relation inventories were completed (§7). We score sentence graphs with AnCast++ (Sun and Xue, 2024); document-level relations are outside this study.

Data. For PARSEME we use the official files: Chinese train (35,326 sentences), dev and test (3,462 sentences, 786 VMWEs, 38% of them unseen in train); English train (3,471) and test (3,965 sentences, 501 VMWEs). Reflection reads 5-sentence batches from the training file, three sentences with a VMWE and two without, so that every batch has something to learn from; the development set is 150 sentences in their natural proportion (from the official dev file for Chinese, held out of train for English, which has no dev file). For UMR we use the 31 Chinese gold documents and the native UMR 2.0 English annotation, excluding the English sentences that were converted only in part from other banks. Chinese: batches from documents 0004–0014 (taken round-robin over the documents), a development set of 27 sentences, and a test set of eight documents (138 sentences) that earlier analyses of this project had looked at, so the Chinese UMR comparison is retrospective. English: batches from the guideline's example sentences and a narrative document, 25 development sentences, and a test set of five news documents and a narrative passage (143 sentences). Twelve rounds read 60 training sentences.

5 Experimental Setup

Models. Closed decisions are answered by Jev (`jev-1.13-20260917`) through OpenRouter. The reflector is Claude Sonnet 5.5 and the cascade model Claude Haiku 5.5, both through OpenRouter at temperature 0 (0.3 when the reflector's first reply cannot be parsed). We report the costs that the service billed for each request.

Reflection. Every run has twelve rounds of five training sentences (60 sentences in all). The reflector proposes at most three edits per round and may rewrite the rejected ones once. An edit is kept under the measured criterion of §3.2. Before an edit is tried, it is checked for text copied from the test sentences (six consecutive Chinese characters or four English words); none was found. The development and test sets are never shown to the reflector. Seeds and every round's version are frozen, and all reported test runs use a version that existed before any test sentence was parsed with it.

Baselines. (i) *Direct generation*: Claude Haiku 5.5 writes the annotation itself, one call per sentence, given the scheme's category or concept inventory and the first ten sentences of the reflection pool with their gold annotation; it thus sees one sixth of the gold that reflection reads, in full, at every call. (ii) For PARSEME, the systems of the shared tasks, trained on the full training data and in the open track on pretrained models, re-scored from their released test predictions with the official script and the same training file. (iii) For Chinese UMR, two closed procedures from earlier work on these data: the seed J2 and K4, the latter after four rounds of document-level measured reflection with a different reflector (seven training documents in all).

Metrics. PARSEME: the official MWE-based F1 (exact token set and category) and its restriction to VMWEs unseen in the training file. UMR: AnCast++ sentence-graph F; each test document is scored on its sentences with gold, and we average over documents. In a pilot on these data, a cold re-run of the same Jev procedure changed a document's UMR F by at most about 0.6; we treat smaller differences as noise.

6 Results

Learned procedures beat direct generation on PARSEME. Table 1 gives the PARSEME results. The seed procedures, written from the guidelines before any training sentence was read, already exceed direct generation with ten gold examples: 52.4 vs. 39.5 F in Chinese and 44.3 vs. 41.1 in English. Reflection adds 5.5 F in Chinese (7.8 on unseen expressions) and 2.0 in English. In English, the learned procedure scores 13 points above the best system of the 2018 shared task, which was trained

System		training sentences	Chinese (1.2) F	Chinese (1.2) unseen F	English (1.1) F	English (1.1) unseen F
Shared-task systems, trained on the full training data (re-scored)						
best open track		all	71.9	64.2	33.3	23.7
best closed track		all	60.9	45.7	32.9	17.6
Direct generation						
Claude Haiku 5.5, 10 gold examples		10	39.5	32.6	41.1	34.8
Decision orchestration (Jev decisions)						
seed procedure	Π_0	0	52.4	36.5	44.3	37.8
after 6 rounds	Π_6	30	56.9	43.1	**46.3**	**38.8**
after 12 rounds	Π_{12}	60	57.8	44.3	**46.3**	**38.8**
Π_{12} + Haiku cascade (τ=0.7)		60	**58.5**	**45.7**	44.1	37.4

Table 1: PARSEME test sets: MWE-based F1 and F1 on VMWEs unseen in the training file, all computed with the official script from the systems' released predictions. Best open track: TRAVIS-mono (Chinese), Milos (English); best closed track: ERMI, TRAPACC. "Training sentences" are the annotated sentences a system used; the reflector read five per round.

on all 3,471 training sentences. Those systems predate today's pretrained encoders, so the comparison shows how little data the procedure needs rather than a new state of the art. In Chinese, the open-track systems trained on 35,326 sentences on top of multilingual pretrained encoders remain 14 points ahead, and the procedure is 3 points below the best closed-track system. Precision is the weak side of both procedures: the final Chinese procedure finds 71% of the gold VMWEs, but only 49% of its predictions are correct.

Routing helps where Jev is unsure and the cheap model is right. In Chinese, the cascade re-asked 361 questions ($0.09) and gained 0.7 F (1.4 on unseen expressions). In English it re-asked 720 questions and lost 2.2 F: Haiku accepted 123 more candidates as VMWEs, of which 13 were correct. Whether to route is an empirical question per task, which a development set can answer before deployment.

On UMR, closed decisions are not enough. Table 2 gives the UMR results. The best Chinese procedure with routing reaches 42.6 F against 42.0 for direct generation; in English, direct generation is 16 points ahead. Reflection still helps where the seed is far from the data: the procedure written for Chinese gains 4.1 F on English in twelve rounds, learning English conventions such as contracted negation, clitic pronouns, plural nouns and names of people written as `person`. In Chinese, the same twelve rounds raised the batches by several points but left the test score unchanged. Routing uncertain decisions to Haiku adds 2.5 F to K4 at τ=0.7, but only 1.5 at τ=0.9: Haiku is better than

System	zh	en
Haiku 5.5, 10 gold examples	42.0	**46.4**
Chinese procedures of earlier work		
J2 (seed)	39.4	–
K4 (J2 + 4 document rounds)	40.1	–
K4 + Haiku cascade, τ=0.7	**42.6**	–
K4 + Haiku cascade, τ=0.9	41.6	–
Revised procedure (questions in data files)		
as released (27 aspect values)	33.3	–
Π_0: + aspect / polarity conventions	38.5	26.0
Π_6	38.2	29.7
Π_{12}	38.2	30.1
Π_{12} + Haiku cascade, τ=0.7	–	30.5

Table 2: UMR test sets: AnCast++ sentence F, macro-averaged over documents (Chinese: 8 documents, 138 sentences; English: 6, 143). The English procedure is the Chinese Π_0 reflected on English batches.

Jev on Jev's most uncertain questions, not on all of them. Parsing the Chinese test set costs $0.40–0.82 in decisions, plus $1.2–1.5 for the cascade; direct generation costs $0.19.

Reflection is cheap and mostly edits rules. Table 3 summarizes the reflection runs. Twelve rounds cost $0.53–4.27 in reflector calls; decisions during reflection cost at most $0.74, because unchanged questions are answered from the cache. Of 190 proposed edits, 83 were kept: 62 of 146 rule edits, 9 of 17 threshold changes, 6 of 11 new or changed options and 6 of 16 rewordings. The reflector mostly proposes rules. On PARSEME, 26 of the 37 kept edits change the candidate rules (a new candidate kind, a filter for literal particles, an exception list) and 11 change the decision code or its thresholds.

Run	tried	kept	dev Δ	test Δ	$ refl.
PARSEME zh	34	20	+7.6	+5.5	0.53
PARSEME en	39	17	+2.9	+2.0	0.69
UMR zh (from Π_0)	42	15	+0.6	−0.3	4.27
UMR zh (from J0)	32	18	+7.3	+1.6	2.27
UMR en	43	13	+7.5	+4.1	3.41

Table 3: Twelve rounds of reflection (ten for UMR zh from J0, an earlier hand-written seed whose development set had ten sentences): edits tried and kept, the change on the development and test sets, and the reflector's cost. Decisions during reflection cost $0.01–0.74 per run.

7 Analysis

Options must follow the annotation, not the inventory. The revised UMR procedure moved every question into data files and completed its inventories from the guidelines, among them the 27 aspect values of the UMR aspect lattice. The gold annotation uses seven of them. Given the full lattice, Jev chose fine-grained values the gold never uses (*directed-achievement* 118 times, *irreversible-directed-achievement* 59 times on the Chinese test set), the aspect score of the scorer's fine-grained judge fell from 0.70 for the procedure it revised to 0.31, and F fell to 33.3. Restricting the options to the seven values of the gold (aspect 0.71), together with one polarity rule the revision had dropped, gave 38.5 (Table 2). A decision model answers the question it is asked; whether the options are the right ones is a property of the procedure, and only measurement against gold reveals it. In the same spirit, we tried asking aspect in two steps (a coarse kind, then the value within it): with the same decision model and test set, aspect accuracy dropped from 0.72 to 0.67 and F by 0.3, so we kept the single question.

Where UMR loses: structure, not labels alone. To see what closed decisions leave on the table in UMR, we replaced parts of K4's test output by gold, using the scorer's alignment of predicted to gold nodes, and re-scored. Making every concept form, aspect, role label and attribute of the aligned nodes correct raises F from 40.4 to 49.0; additionally making the core and peripheral arguments among aligned nodes correct raises it to 55.9. The remaining gap is nodes that the procedure never built. Correct aspect values alone are worth 2.9 F, but they are the least learnable part: a lexical prior from the training gold does not beat Jev on the test set (0.714 vs. at most 0.716 accuracy), since the same verbs receive different aspects across documents. Attachment is the procedure's own work: rules split a sentence top-down and assign each word to a role by a per-word question, and an early routing error moves a whole subtree. On PARSEME, by contrast, the structure is a set of token groups proposed by rules, and every remaining error is a decision or a missing candidate rule, which is exactly what reflection edits.

The development set decides whether reflection transfers. The three kinds of evidence a reflection round uses (the batch, earlier batches and the development set) protect against different failures. On PARSEME, with 150 development sentences, development gains carried over to the test set (+7.6 dev, +5.5 test in Chinese). On Chinese UMR, a first run (from the earlier seed J0) with a ten-sentence development set accepted 18 of 32 edits and raised development F by 7.3 but test F by only 1.6. A second run (from Π_0) with 27 development sentences accepted 15 of 42 edits; development F rose by 0.6 and test F did not change. The seeds differ, so the comparison is suggestive, but the pattern is the expected one. A small development set accepts edits that fit its own sentences; a larger one rejects them, which is the intended behavior but leaves the batches' gains behind. Measurement makes this visible: every kept edit has a recorded effect on each set.

What the procedure costs. Decisions are the cheap part. Deciding the 3,462 sentences of the Chinese PARSEME test set cost $0.06; a UMR sentence takes 11–49 requests to Jev and costs $0.001–0.006, depending on the procedure. Reflection costs a few dollars per run and is paid once. Routing costs more per decision than Jev by two orders of magnitude, but only for uncertain questions (Chinese PARSEME: $0.09 for the test set). Direct generation by a cheap LLM costs $0.11–1.16 per test set, more than the procedure's decisions on PARSEME and less than the procedure on UMR.

8 Conclusion

We wrote annotation guidelines as procedures that ask a decision model closed questions and learned them from five annotated sentences at a time, keeping only edits that measurably help. Where the guideline is a sequence of tests over candidates that rules can enumerate, as for verbal multiword expressions, sixty annotated sentences and a few

dollars give a procedure that beats direct LLM annotation by 5–18 F and, in English, the supervised systems of the shared task. Where the output is mostly structure, as in UMR, the closed interface is the bottleneck: routing and reflection help, but direct generation is as good or better. The procedures, their edit histories and their decision logs are readable artifacts, and they show where a scheme's conventions depart from its guideline, as the aspect inventory did. Future work should let a procedure ask structural questions as reliably as categorical ones, choose the routing threshold on development data, and test the method with annotators in the loop.

Limitations

We test two annotation schemes in two languages each, with one run per configuration; repeated runs and other document orders would show the variance of reflection, which we only bound for decision noise. The Chinese UMR test set had been inspected in earlier analyses of this project, so its comparisons are retrospective; the English UMR and both PARSEME test sets were not used before the final runs, except that the seed's candidate coverage was computed once on the Chinese PARSEME test set, without changing the seed. PARSEME English comes from edition 1.1, whose systems predate pretrained encoders; our comparison with them shows data efficiency, not a new state of the art, and edition 1.2 open-track systems remain far ahead in Chinese. The procedure reads the lemmas, parts of speech and dependency relations provided with the PARSEME data. The seed procedures were written by the authors from the guidelines with knowledge of the task, and the earlier UMR procedures used training documents for hand-tuning; their gold exposure is reported with each version. Jev is a commercial service with a version identifier, and LLM behavior may change; we release every version of the procedures, their edit histories with the measured effect of each edit, and the test-set predictions. Costs are list prices billed by one provider at the time of the experiments.

Ethics Statement

We use publicly released annotated corpora under the licenses of their releases (e.g., PARSEME 1.2 Chinese is CC BY-NC-SA 4.0). No new human annotation was collected and no personal data were processed. Procedures that annotate cheaply could be used to produce training data at scale; their errors are systematic and readable, which makes them easier to audit than those of a direct generator, but outputs should still be checked before they are used as gold. LLMs were used as components of the method, as described, and to assist with code and writing; the authors are responsible for all content.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*.

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. *arXiv preprint arXiv:2203.07836*.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Julia Bonn, Matthew J. Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benet Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Julia Bonn, Chen Ching-wen, James Andrew Cowell, William Croft, Lukas Denk, Jan Hajič, Kenneth Lai, Martha Palmer, Alexis Palmer, James Pustejovsky, Haibo Sun, Rosa Vallejos Yopán, Jens

Van Gysel, Meagan Vigus, Nianwen Xue, and Jin Zhao. 2023. Uniform meaning representation. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

Jayeol Chun and Nianwen Xue. 2024. A pipeline approach for parsing documents into uniform meaning representation graphs. In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14953–14962.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, and 1 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696, Online. Association for Computational Linguistics.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, and 1 others. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *The Eleventh International Conference on Learning Representations*.

Minzhi Li, Taiwei Shi, Caleb Ziems, Min-Yen Kan, Nancy Chen, Zhengyuan Liu, and Diyi Yang. 2023. CoAnnotating: Uncertainty-guided work allocation between human and large language models for data annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1487–1505, Singapore. Association for Computational Linguistics.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2025. Setup: Sentence-level english-to-uniform meaning representation parser. *arXiv preprint arXiv:2512.07068*.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA. Association for Computational Linguistics.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Carlos Ramisch, Silvio Ricardo Cordeiro, Agata Savary, Veronika Vincze, Verginica Barbu Mititelu, Archna Bhatia, and 1 others. 2018. Edition 1.1 of the PARSEME shared task on automatic identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Linguistic Annotation, Multiword Expressions and Constructions (LAW-MWE-CxG-2018)*, pages 222–240, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Manon Scholivet, Agata Savary, Carlos Ramisch, Eric Bilinski, Takuya Nakamura, Maria Mitrofan, and Vasile Pais. 2026. Edition 2.0 of the PARSEME shared task on multilingual identification and paraphrasing of multiword expressions. In *Proceedings of the 22nd Workshop on Multiword Expressions (MWE 2026)*, pages 254–275. Association for Computational Linguistics.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let me speak freely? a study on the impact of format restrictions on large language model performance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, Miami, Florida, US. Association for Computational Linguistics.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

TypeSafe. 2026. Jev API reference. `https://docs.typesafe.ai/api`. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(0):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. 2024. Human-LLM collaborative annotation through effective verification of LLM labels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–21, Honolulu, HI, USA. ACM.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*, pages 1–22, New Orleans, LA, USA. ACM.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR.

Jiawei Zhou, Tahira Naseem, Ramón Fernandez Astudillo, Young-Suk Lee, Radu Florian, and Salim Roukos. 2021. Structure-aware fine-tuning of sequence-to-sequence transformers for transition-based AMR parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6290, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A Examples of Kept Edits

Each edit is an exact find-and-replace in one file of the procedure, proposed by the reflector with a reason and kept because its measured effect met the criterion of §3.2. Gains are F on the batch, the regression set and the development set.

PARSEME English, round 6 (`candidates.py`; batch +14.3, regression ±0, dev +1.9). *Reason given:* "*V as NP* marks a role or function (*acts as, serve as, work as*) and is not an IAV; removing it avoids false positives of this kind generally."

```
- ... not in PARTICLES | {"to"}
+ ... not in PARTICLES | {"to", "as"}
```

PARSEME Chinese, round 3 (`candidates.py`; batch +7.3, regression ±0, dev +0.7). *Reason given:* "A causative 讓/使/令, or 有, followed by a verb or adjective is a clausal complement, not a light verb plus event noun." The light-verb candidate rule gains the condition `and not (h.form in {讓, 让, 使, 令, 有} and t.upos != NOUN)`.

UMR English, round 9 (`jevlib.py`; batch +6.1, regression ±0, dev +1.1). *Reason given:* "Contracted pronouns such as *he's*, *you're* and *they'll* are recognized as pronouns. They then get a person node with `:refer-person` and `:refer-number` instead of a literal concept like `he's`."

```
+ elif lang == "en":
+   for s in ("'s","'ll","'d","'m","'re","'ve"):
+     if (t.endswith(s) and
+         t[:-len(s)] in PRONOUNS["en"]):
+       t = t[:-len(s)]
+       break
```

B The PARSEME Seed Procedures

Candidates. Chinese: (1) a verb, auxiliary, adjective or noun token of two or more Han characters ending in a directional, resultative or co-verb complement from a fixed list (上, 下, 出, 起, 開, 完, 好, 掉, 住, 到, 為, 成, 在, 入, 至, 給, ...), read as stem + complement; (2) a four-character Han token; (3) a verb and a complement token attached to it (`mark`, `compound:vv`, ...) within three tokens, or adjacent to it; (4) a light or causative verb from a list of 60 (進行, 提供, 作出, 受到, 帶來, 造成, 引起, ...) and its object. English: (1) a verb and its particle (`compound:prt`, or a particle word as `advmod` within three tokens); (2) a light verb from a list of 45 (*make, take, give, have, do, pay,* ...) and its object noun; (3) a verb and the preposition right after it; (4) *make, let, get, break, come, go* with a bare complement.

Questions. One choice per candidate, with options by kind. Complement candidates (Chinese): VPC.semi, MVC, none. Light-verb candidates: LVC.full, LVC.cause, VID, none. Four-character tokens: a yes/no question ("a fixed idiom used

as a verb or predicate here"). Particle candidates (English): VPC.full, VPC.semi, none; preposition candidates: IAV, VPC.full, none; complement candidates: VID, none. Every option carries one or two sentences from the guideline with examples. A candidate is kept when its most probable option is a category with $p \geq 0.5$; overlapping candidates are resolved by probability.

After reflection. The Chinese procedure grew from 64 to 80 lines of candidate rules and from 78 to 95 lines of decision code; the English one from 44 to 83 and from 69 to 96. New rules include exception lists (complements after perception verbs, causatives followed by a clause), new candidate kinds (English *verb* + *particle* + *of*, idiom candidates *verb* + *preposition* + *noun*), and per-verb thresholds (a stricter one for *get* + noun).

C The Reflector's Prompt

The system prompt describes the procedure (skills that split the input by rules, ask Jev closed questions and assemble the answers by rules) and ranks what makes a good edit: the correct answer must be among the options and a rule must turn it into the gold structure; a closed question is better than a rule that guesses; long or ambiguous option lists should be split or given a *none* option; thresholds are constants to change; gold conventions belong in rules; rewording only when an option is clearly misread. Edits must be general (no special-casing of a sentence from the report), keep the code runnable, and not depend on each other. The user prompt contains the task description, the full text of the editable files, the titles of edits tried in earlier rounds with their outcome, and the batch report: for each sentence, the gold and predicted annotations, the scorer's findings, and for every node the questions asked with Jev's top options and probabilities. The reflector answers in JSON with a diagnosis and at most three edits. Rejected edits are returned once with their measured effect on each set and the reason for rejection.

D Runs and Costs

Item	requests	$
Reflection (Sonnet requests; $ Sonnet / Jev)		
PARSEME zh	18	0.53 / 0.01
PARSEME en	22	0.69 / 0.01
UMR zh (Π_0)	23	4.27 / 0.74
UMR en	22	3.41 / 0.24
Test sets		
PARSEME zh test, Π_0, Jev	1,448	0.06
PARSEME zh test, cascade	361	0.09
PARSEME en test, cascade	720	0.14
UMR zh test, K4, Jev	6,500	0.75
UMR zh test, K4, cascade	2,554	1.18
Direct, PARSEME zh / en	3,462 / 3,965	1.16 / 0.85
Direct, UMR zh / en	138 / 143	0.19 / 0.11

Table 4: Billed costs. Decision costs during reflection are low because unchanged questions are answered from the cache.

E Reproduction

The code, data splits, seed procedures, every version Π_r with the round's report, proposals, trials and decision, and all test-set predictions are in the accompanying repository (`rsi-batch/` for the driver and the two task adapters, `experiment/decision-orchestration/` for the runs, with `summary.md` listing every number in this paper). A run is started with `rsi-batch/run_parseme.sh LANG 12` or `rsi-batch/run_umr_v11.sh LANG 12`, and tested with `rsi-batch/evaluate_test.py`.